

Analyzing Message-Code Inconsistency in AI Coding Agent-Authored Pull Requests

Jingzhi Gong
King's College London
London, United Kingdom
jingzhi.gong@kcl.ac.uk

Giovanni Pinna
University of Trieste
Trieste, Italy
giovanni.pinna@phd.units.it

Yixin Bian
Harbin Normal University
Harbin, China
bianyixin@hrbnu.edu.cn

Jie M. Zhang*
King's College London
London, United Kingdom
jie.zhang@kcl.ac.uk

Abstract

Pull request (PR) descriptions generated by AI coding agents are the primary channel for communicating code changes to human reviewers. However, the alignment between these messages and the actual changes remains unexplored, raising concerns about the trustworthiness of AI agents. To fill this gap, we analyzed 23,247 agentic PRs across five agents using PR message-code inconsistency (PR-MCI). We contributed 974 manually annotated PRs, found 406 PRs (1.7%) exhibited high PR-MCI, and identified eight PR-MCI types, revealing that *descriptions claim unimplemented changes* was the most common issue (45.4%). Statistical tests confirmed that high-MCI PRs had 51.7% lower acceptance rates (28.3% vs. 80.0%) and took 3.5× longer to merge (55.8 vs. 16.0 hours). Our findings suggest that unreliable PR descriptions undermine trust in AI agents, highlighting the need for PR-MCI verification mechanisms and improved PR generation to enable trustworthy human-AI collaboration.

CCS Concepts

• **Software and its engineering** → **Collaboration in software development**.

Keywords

LLMs, Code Review, Human-AI Collaboration, AI Trustworthiness, Empirical SE, Mining Software Repositories, LLM4Code, AI4SE

ACM Reference Format:

Jingzhi Gong, Giovanni Pinna, Yixin Bian, and Jie M. Zhang. 2026. Analyzing Message-Code Inconsistency in AI Coding Agent-Authored Pull Requests. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793583>

1 Introduction

AI coding agents are increasingly acting as autonomous teammates in software development [8, 29, 30], generating code and opening pull requests (PRs) with natural-language descriptions that communicate intent [22]. Unlike human-written PRs, AI agent-authored PRs (Agentic-PRs) are largely black boxes [30], forcing reviewers to rely on PR descriptions to understand changes and assess correctness [6, 34]. As a result, the reliability of PR descriptions is critical for enabling effective human-AI collaboration.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/26/04

<https://doi.org/10.1145/3793302.3793583>

However, AI-generated descriptions (title + body) are not always faithful to the underlying code, as generative models can produce hallucinated or incorrect statements [14, 27], undermining trust in AI agents. While message-code inconsistency (MCI) has been studied for human-written commit messages [4, 31, 33], little is known about how often Agentic-PR descriptions misalign with code changes, what inconsistency types occur, or whether such misalignment affects reviewer trust and PR outcomes.

To address this gap, we conducted an empirical study analyzing 23,247 Agentic-PRs from the AIDev dataset [17] using *PR message-code inconsistency* (PR-MCI) and made the following contributions:

(1) **Annotated dataset:** We release 974 manually annotated PRs (432 partial/misaligned cases), supporting the development of PR-MCI detection methods and more reliable AI coding agents.

(2) **RQ1: How often do Agentic-PR descriptions misalign with their code changes?** We found 406 high-MCI PRs (1.7%) with a 20-fold variation across agents, suggesting that a non-trivial fraction of PRs could mislead reviewers.

(3) **RQ2: What types of message-code inconsistencies occur in AI-generated PRs?** We identified eight PR-MCI types, finding that *Phantom Changes* (descriptions claim unimplemented changes) dominated (45.4%).

(4) **RQ3: Is message-code inconsistency associated with PR acceptance and review effort?** We showed that high-MCI PRs had 51.7% lower acceptance rates (28.3% vs. 80.0%) and took 3.5× longer to merge (55.8 vs. 16.0 hours).

Overall, our findings indicate that while AI agents can generate PRs at scale, the reliability of their descriptions varies substantially and is associated with reviewer trust and PR outcomes, offering actionable insights for developers, tool builders, and researchers.

2 Background and Related Work

Agentic-PRs. AI coding agents are increasingly capable of autonomously authoring and submitting PRs in real-world software projects [2, 22, 30]. In these *Agentic-PRs*, natural-language descriptions play a critical role in communicating intent and scope to human reviewers. However, generative AI systems are known to produce hallucinated or incorrect statements [14, 27], raising concerns about the reliability and trustworthiness of Agentic-PRs.

Message-Code Inconsistency. Commit messages and PR descriptions serve as documentation of developer intent. When these descriptions misalign with actual code changes, they create *message-code inconsistency* [4, 28]. Recent work has studied MCI in human-written commit messages and benchmarked LLMs for detecting text-code inconsistencies [4, 33], highlighting challenges in generating faithful change descriptions.

Gap in PR-MCI. While MCI has been studied for human-written commit messages [4, 33], and prior work has analyzed PR text for

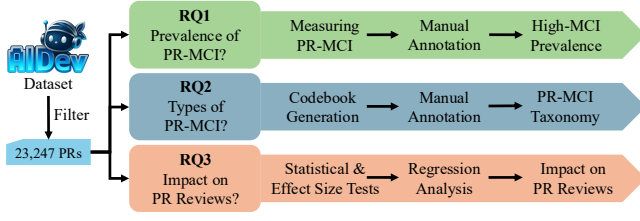


Figure 1: Methodology workflow for PR-MCI analysis.

quality concerns [16], very few prior works examine inconsistency in Agentic-PR descriptions¹. This gap is critical because agents generate descriptions automatically without human oversight, potentially producing misleading explanations at scale. PR decision-making research shows that description quality is associated with acceptance rates [34], and automatically created PRs can differ in how maintainers interact with them [32]. Our work extends this research by examining how description-code consistency is associated with outcomes for Agentic-PRs.

3 Methodology and Experiment Setup

Figure 1 illustrates our workflow. We used the *AIDev-pop* subset (33,596 PRs from 2,807 repositories with >100 stars) from the *AIDev* dataset [17]. After filtering for closed PRs, permissive licenses (MIT or Apache 2.0), and six most common task types (95.2%), the final dataset contains **23,247 PRs** authored by five AI agents.

3.1 Measuring PR-MCI (RQ1)

Following prior MCI research [31, 33], we used PR-MCI as the degree to which a PR description (title + body) diverges from the underlying code changes. We measured PR-MCI using a **heuristic similarity score** that combines three complementary signals: *scope adequacy* s_s (whether description verbosity matches code churn), *file-type consistency* s_f (whether mentioned file types, e.g., tests or documentation, are actually modified), and *task-type alignment* s_t (whether description language matches the labeled task type). The similarity score $s \in [0, 1]$ aggregates these signals using a weighted scheme [15]: $s = 0.3 \cdot s_s + 0.4 \cdot s_f + 0.3 \cdot s_t$, where s_f receives the highest weight (40%) for providing the most concrete and verifiable evidence of misalignment (e.g., claiming test changes when no test files are modified), while s_s and s_t each receive 30% as complementary but less definitive signals [1]².

To validate the PR-MCI score and calibrate the decision threshold, two authors **manually annotated** 600 PRs, stratified by agent, task type, and score bins, as *aligned*, *partially aligned*, or *misaligned*, achieving strong agreement ($\kappa = 0.892$). PRs with similarity below $\theta = 0.61$ are labeled *high-MCI*, where θ is selected by optimizing F1 on labeled validation data [5] after conducting sensitivity analysis across threshold ranges [18]; five-fold cross-validation shows stable threshold selection (mean=0.606, std=0.008) [33]. We also evaluated alternative methods: an embedding model (Qwen3-Embedding-0.6B [24]) performs poorly (F1=0.150), and an agreement method (both methods must agree) achieves lower F1 (0.567 vs. 0.630), indicating both are insufficient (see Table 1). Consistent with large-scale

¹Note that PR descriptions differ from commit messages: PR descriptions may span multiple commits and serve as the primary communication channel for reviewers.

²Component ablation analysis is available in our [supplementary material](#).

Table 1: High-MCI Prevalence and Validation Metrics. [lower, upper] show 95% Wilson confidence intervals. **Highlighted values are from the primary Heuristic MCI scoring method.**

Category	Heuristic (PRIMARY)	Embedding	Agreement
Overall Prevalence	1.7% [1.6, 1.9]	60.0%	1.4%
High-MCI Prevalence by Agent (%)			
GitHub Copilot	8.7% [7.7, 9.9]	83.5%	7.1%
Cursor	4.5% [3.2, 6.4]	76.8%	3.4%
Claude Code	0.9% [0.2, 3.3]	74.5%	0.9%
OpenAI Codex	0.8% [0.6, 0.9]	52.7%	0.6%
Devin	0.4% [0.2, 0.7]	75.5%	0.2%
High-MCI Prevalence by Task Type (%)			
Chore	4.0% [2.7, 5.9]	54.5%	3.0%
Refactor	3.5% [2.7, 4.5]	47.7%	2.2%
Bug Fix	2.1% [1.8, 2.5]	49.5%	1.6%
Feature	1.5% [1.3, 1.8]	69.7%	1.3%
Documentation	1.0% [0.7, 1.5]	57.4%	0.8%
Test	1.0% [0.6, 1.5]	49.3%	0.6%
Validation Metrics (n=600)			
Precision	0.742	0.083	0.760
Recall	0.548	0.786	0.452
F1	0.630	0.150	0.567

inconsistency studies [31], we report prevalence with **95% Wilson confidence intervals**.

3.2 PR-MCI Taxonomy Development (RQ2)

Following mixed-method inconsistency studies that derive taxonomies from manually coded, automatically retrieved candidates [31], we developed a PR-MCI taxonomy through an **iterative process**: GPT-5.2 [23] first generates an initial codebook with eight categories, which two annotators refine through discussion before **manually classifying** 432 partial/misaligned PRs from 974 PRs (600 validation + 374 additional high-MCI PRs from RQ1).

Note that RQ1 and RQ3 use a *strict* binary definition of high-MCI (treating *partially aligned* PRs as aligned) to identify only clearly misaligned PRs, whereas RQ2 includes both partial and misaligned PRs to capture a broader range of inconsistency patterns for a more comprehensive taxonomy. As in prior work, the taxonomy reflects the analyzed subset and may not capture all inconsistency types.

3.3 Statistical Tests (RQ3)

To examine associations between PR-MCI and outcomes, we compared high- and low-MCI PRs using standard statistical tests. Following PR decision research [34], we analyzed acceptance rate, time to merge, review count, and comment count. For binary outcomes (acceptance), we used **Chi-square tests** [20] with **Cramér's V** [3]; for continuous variables (e.g. time), **Mann-Whitney U tests** [21] with **Cliff's δ** [19]. Significance is assessed at $p < 0.05$.

To control for confounders, we fitted **regression models** including log-transformed code churn, files changed, task type, and agent, following established PR outcome research [34].

4 Results

4.1 RQ1: Prevalence of PR-MCI

Table 1 reports prevalence and validation metrics. Overall, **406 out of 23,247 Agentic-PRs (1.7%)** exhibited high PR-MCI (similarity score below 0.61), where descriptions poorly match code

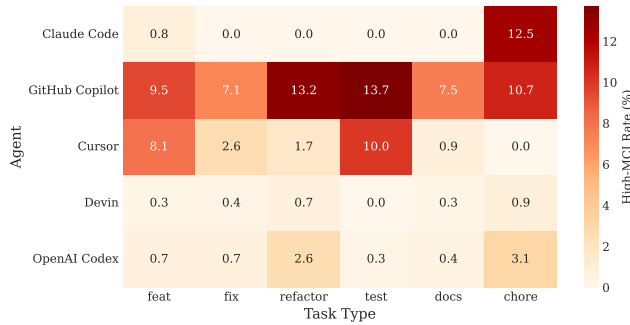


Figure 2: Heatmap of high-MCI prevalence (%) by agent×task.

changes, suggesting that a small but non-trivial fraction could mislead reviewers. The prevalence revealed a **20-fold difference** across agents: GitHub Copilot had the highest rate (8.7%, 234 high-MCI PRs out of 2,675), followed by Cursor (4.5%, 31 out of 682).³

Across task types, we observed a **4-fold variation**: chore PRs had the highest inconsistency rate (4.0%, 24 out of 600), followed by refactoring (3.5%, 54 out of 1,553) and bug fixes (2.1%, 113 out of 5,319). Features (1.5%), documentation (1.0%), and test PRs (1.0%) showed the lowest rates. The heatmap (Figure 2) reveals agent×task interactions: GitHub Copilot showed elevated rates across most task types, particularly for test (13.7%) and refactor tasks (13.2%).

RQ1: 406 PRs (1.7%) exhibited high PR-MCI, suggesting that a non-trivial fraction of PRs could mislead reviewers. In particular, GitHub Copilot showed 8.7% (20x higher than Devin’s 0.4%), and chore tasks showed 4.0% (4x higher than test tasks’ 1.0%).

4.2 RQ2: PR-MCI Taxonomy

Figure 3 shows the taxonomy over all, by agent, and by task type. In particular, the most common PR-MCI type was **Phantom Changes (45.4%)**, where PR descriptions claim unimplemented changes. This was followed by Scope Understated (22.0%), where descriptions omit significant changes, and Placeholder/Incomplete (18.8%), which used generic or boilerplate text⁴.

For agent-specific patterns, GitHub Copilot was dominated by Phantom Changes (74.0%), while Scope Understated was the primary issue for Cursor (52.8%) and Devin (41.0%). OpenAI Codex showed a balanced distribution with Placeholder/Incomplete (39.8%) and Scope Understated (41.9%).

Task-specific patterns showed that Test (55%) and Bug Fix (53.5%) PRs were most prone to Phantom Changes, while Refactor PRs showed Scope Understated (40.5%) as the dominant issue.

RQ2: Phantom Changes was the dominant PR-MCI type (45.4%), followed by Scope Understated (22.0%) and Placeholder/Incomplete (18.8%), suggesting that AI agents frequently claim unimplemented changes or omit significant changes.

4.3 RQ3: Impact on Review Outcomes

Table 2 reports summary statistics for key metrics with statistical tests and breakdowns by agent and task type. Notably, high-MCI

³Sensitivity analysis (threshold 0.55-0.65) suggested that relative ordering remained consistent across thresholds, as in our [supplementary material](#).

⁴Examples of different PR-MCI types are provided in our [supplementary material](#).

Table 2: Review and Outcome Metrics by MCI Level. Highlighted values indicate statistically significant differences between High-MCI and Low-MCI PRs with non-negligible effect sizes. See §3 for statistical test details.

Metric	Low-MCI	High-MCI	p-value	Effect Size	# Low	# High
Acceptance Rate (%)	80.0	28.3	<0.001	Small	22,841	406
Mean Time to Merge (h)	16.0	55.8	<0.001	Small	18,282	115
Mean # Review	0.74	0.42	0.039	Negligible	22,841	406
Mean # Comment	0.64	0.37	0.012	Negligible	22,841	406
Acceptance Rate by Agent (%)						
Claude Code	69.3	50.0	1.000	Negligible	218	2
GitHub Copilot	59.4	3.4	<0.001	Medium	2,441	234
Cursor	73.3	67.7	0.638	Negligible	651	31
Devin	58.3	45.5	0.580	Negligible	2,880	11
OpenAI Codex	87.2	62.5	<0.001	Negligible	16,651	128
Acceptance Rate by Task Type (%)						
Chore	80.2	45.8	<0.001	Small	576	24
Documentation	89.8	34.5	<0.001	Small	2,786	29
Feature	80.1	27.7	<0.001	Small	10,782	166
Bug Fix	73.6	17.7	<0.001	Small	5,206	113
Refactor	78.1	42.6	<0.001	Small	1,499	54
Test	84.3	25.0	<0.001	Small	1,992	20
Time to Merge (h) by Agent (merged PRs only)						
Claude Code	31.4	6.1	0.817	Negligible	149	1
GitHub Copilot	70.2	24.0	0.163	Small	1,420	8
Cursor	23.1	28.9	0.333	Negligible	477	21
Devin	21.8	14.4	0.976	Negligible	1,669	5
OpenAI Codex	5.7	38.6	<0.001	Medium	14,509	77
Time to Merge (h) by Task Type (merged PRs only)						
Chore	16.5	81.4	0.181	Small	460	11
Documentation	8.9	1.1	0.140	Small	2,493	10
Feature	11.3	16.4	<0.001	Medium	8,617	44
Bug Fix	20.6	34.4	<0.001	Medium	3,812	20
Refactor	15.5	51.2	0.012	Small	1,165	22
Test	6.4	80.6	0.825	Negligible	1,677	5

PRs were associated with a significantly lower acceptance rate: **28.3% vs. 80.0%**, a difference of 51.7% ($p < 0.001$). The corresponding effect size was small (Cramér’s $V = 0.166$), indicating a non-trivial practical difference. The impacts on acceptance rate varied substantially by agent: for GitHub Copilot, the effect was largest (3.4% vs. 59.4%, a 55.9 percentage point drop, $p < 0.001$). The effect of high-MCI was even more obvious by task type, with statistically significant differences for all tasks (all $p < 0.001$ with small effect sizes).

For the time to merge, high-MCI PRs took significantly longer: **55.8 hours vs. 16.0 hours** ($p < 0.001$; small effect size with Cliff’s $\delta = 0.310$). The impact was most pronounced for OpenAI Codex (38.6 vs. 5.7 hours, $p < 0.001$) and refactoring tasks (51.2 vs. 15.5 hours, $p = 0.012$).

Further, regression analysis⁵ showed that PR-MCI remained significantly associated with lower acceptance and longer merge times ($p < 0.001$ for both) after controlling for code churn, files changed, task type, and agent, suggesting that PR-MCI has an independent association with outcomes beyond what can be explained by code complexity or task characteristics.

RQ3: High-MCI PRs had 51.7% lower acceptance (28.3% vs. 80.0%) and took **3.5× longer to merge** (55.8 vs. 16.0 h), confirmed by statistical tests with confounders controlled, suggesting that PR-MCI is associated with reviewer trust and PR outcomes.

⁵Detailed regression results are available in our [supplementary material](#).

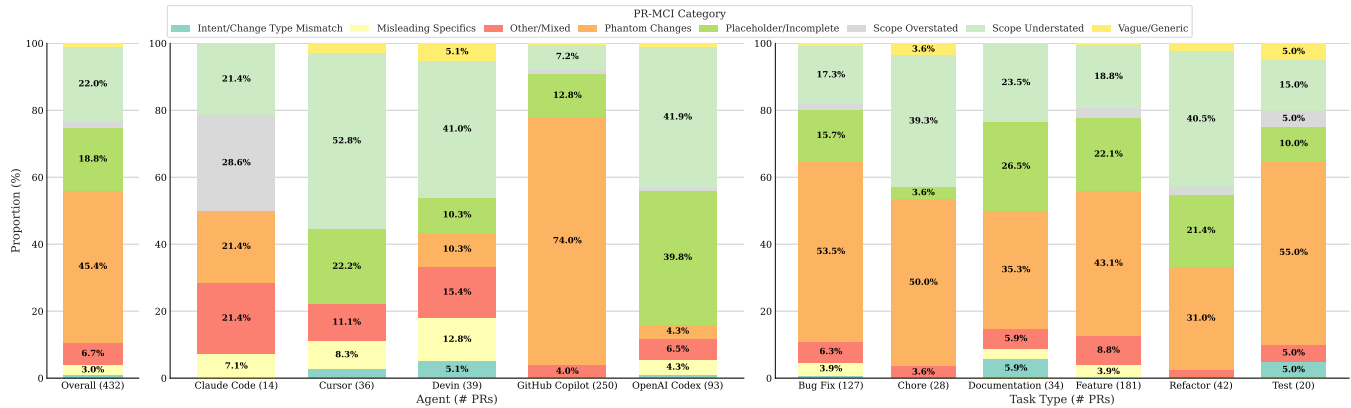


Figure 3: Distribution of PR-MCI categories overall, by coding agent, and by task type.

5 Discussion

5.1 Actionable Insights for Developers

Our findings demonstrate that even when AI-generated code is acceptable, PR descriptions may misstate scope or claim phantom changes, confirming why human oversight remains necessary for Agentic-PRs [30]. Therefore, we suggest:

For agent users: (1) verify that agents actually commit changes, (2) avoid template-based prompts, and (3) refine prompts to guide agents toward more complete descriptions.

For PR reviewers, we recommend three quick heuristic checks to identify PR-MCI without examining the diff in detail:

For PR reviewers: (1) verify the diff is not empty, (2) check if descriptions seem too brief for the amount of code changed, and (3) check for template markers (e.g., [WIP], [Failed], or generic phrases like "I'm starting to work on it").

5.2 Actionable Insights for AI Tool Builders

Our analysis reveals that high-MCI PRs take 3.5× longer to merge, wasting around **40 hours per PR**. Even though only 1.7% of PRs have high-MCI, they may affect trust in the AI-driven software development lifecycle [11]. Therefore, we recommend:

For AI tool builders: (1) implement automated PR-MCI verification mechanisms that can detect common types (e.g., Phantom Changes, Scope Understated, and Placeholder/Incomplete), and (2) integrate them into Agentic-PR generation pipelines to flag problematic descriptions before review, helping recover thousands of hours of wasted review effort.

5.3 Implications for SE 3.0 Research

The emerging "SE 3.0" vision positions AI as active teammates in the software development lifecycle [11, 17], where AI agents are expected not only to generate code, but also to communicate intent, justify changes, and support human decision-making in collaborative workflows. Our findings provide empirical evidence that current Agentic-PR descriptions may vary substantially in quality and can strongly affect review outcomes.

Building on this insight, future research could leverage our 974 manually annotated PRs with 432 partial/misaligned cases to:

For Researchers: (1) investigate why agent communication abilities vary, (2) develop standardized evaluation metrics for PR description quality, and (3) explore how Agentic-PR generation can be improved through better prompt engineering [9, 12], fine-tuning [10, 13], or reinforcement learning [7, 25, 26].

5.4 Threats to Validity

This study has several limitations that should be considered when interpreting the results. (1) **Construct validity:** Our PR-MCI metric is a heuristic proxy for semantic consistency and may miss subtle mismatches or penalize concise but accurate descriptions. (2) **Threshold calibration:** The decision threshold is calibrated on a validation set, which weakens absolute prevalence claims despite observed stability under cross-validation. (3) **External validity:** Dataset filtering choices prioritize scale over strict controls and may affect generalizability. (4) **Sample size imbalance:** Agent sample sizes are imbalanced (e.g., Claude Code has only 220 PRs), limiting the reliability of agent-specific conclusions for smaller samples. (5) **Taxonomy completeness:** The taxonomy is derived from a limited subset of PRs (432 partial/misaligned cases) and may not capture all inconsistency types. (6) **Causal inference:** Our analysis is observational and does not establish causal relationships between PR-MCI and review outcomes. (7) **Ethical considerations:** This study analyzes only publicly available GitHub data from repositories under permissive licenses (MIT or Apache 2.0) and reported findings in aggregate to avoid identifying individuals or projects.

6 Conclusion

We used PR-MCI to quantify alignment between AI-generated PR descriptions and code changes, analyzing 23,247 PRs from five AI agents. Our findings reveal that unreliable PR descriptions (high PR-MCI) are associated with significantly lower PR acceptance rates and longer merge times, indicating that improving the consistency and accuracy of AI-generated PR descriptions is essential for trustworthy human-AI collaboration.

Data Availability. All data, scripts, results, and supplementary materials are available at <https://github.com/gjz78910/PR-MCI>.

Acknowledgments

This work has been supported by the ITEA GENIUS grant (project number 23026).

References

- [1] Jacob G Barnett, Charles K Gathuru, Luke S Soldano, and Shane McIntosh. 2016. The relationship between commit message detail and defect proneness in java projects on github. In *Proceedings of the 13th International Conference on Mining Software Repositories*. 496–499.
- [2] Paul Brookes, Vardan Voskanyan, Rafail Giavrimis, Matthew Truscott, Mina Ilieva, Chrystalla Pavlou, Alexandru Staicu, Manal Adham, Will Evers Hood, Jingzhi Gong, et al. 2025. Evolving Excellence: Automated Optimization of LLM-based Agents. *arXiv preprint arXiv:2512.09108* (2025).
- [3] Harald Cramér. 1999. *Mathematical methods of statistics*. Vol. 9. Princeton university press.
- [4] Jinhao Dong, Yiling Lou, Dan Hao, and Lin Tan. 2023. Revisiting learning-based commit message generation. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 794–805.
- [5] Rong-En Fan and Chih-Jen Lin. 2007. A study on threshold selection for multi-label classification. *Department of Computer Science, National Taiwan University* (2007), 1–23.
- [6] Denae Ford, Mahnaz Behrooz, Alexander Serebrenik, and Chris Parnin. 2019. Beyond the code itself: how programmers really look at pull requests. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS)*. IEEE, 51–60.
- [7] Jingzhi Gong. 2025. Enhancing Trust in Language Model-Based Code Optimization through RLHF: A Research Design. *CHASE DECS* (2025).
- [8] Jingzhi Gong, Yixin Bian, Luis de la Cal, Giovanni Pinna, Anisha Uteem, David Williams, Mar Zamorano, Karine Even-Mendoza, William B Langdon, Hector D Menendez, et al. 2025. GA4GC: Greener Agent for Greener Code via Multi-Objective. In *SSBSE 2025 Challenge Case: Green SBSE*.
- [9] Jingzhi Gong, Rafail Giavrimis, Paul Brookes, Vardan Voskanyan, Fan Wu, Mari Ashiga, Matthew Truscott, Mike Basios, Leslie Kanthan, Jie Xu, et al. 2025. Tuning llm-based code optimization via meta-prompting: An industrial perspective. *ASE Industry Showcase* (2025).
- [10] Jingzhi Gong, Vardan Voskanyan, Paul Brookes, Fan Wu, Wei Jie, Jie Xu, Rafail Giavrimis, Mike Basios, Leslie Kanthan, and Zheng Wang. 2025. Language models for code optimization: Survey, challenges and future directions. *arXiv preprint arXiv:2501.01277* (2025).
- [11] Robin Gröpler, Steffen Klepke, Jack Johns, Andreas Dreschinski, Klaus Schmid, Benedikt Dornauer, Eray Tüzün, Joost Noppen, Mohammad Reza Mousavi, Yongjian Tang, et al. 2025. The Future of Generative AI in Software Engineering: A Vision from Industry and Academia in the European GENIUS Project. *AIWare* (2025).
- [12] Dong Huang, Jianbo Dai, Han Weng, Puzhen Wu, Yuhao Qing, Heming Cui, Zhijiang Guo, and Jie Zhang. 2024. Effilearn: Enhancing efficiency of generated code via self-optimization. *Advances in Neural Information Processing Systems* 37 (2024), 84482–84522.
- [13] Dong Huang, Guangtao Zeng, Jianbo Dai, Meng Luo, Han Weng, Yuhao Qing, Heming Cui, Zhijiang Guo, and Jie M Zhang. 2025. SWIFTCODER: Enhancing Code Generation in Large Language Models through Efficiency-Aware Fine-tuning. *ICML* (2025).
- [14] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [15] Ignacy Kaliszewski and Dmitry Podkopaev. 2016. Simple additive weighting—A metamodel for multiple criteria decision analysis methods. *Expert Systems with Applications* 54 (2016), 155–161.
- [16] Shubhashis Karmakar, Zadia Codabux, and Melina Vidoni. 2022. An experience report on technical debt in pull requests: Challenges and lessons learned. In *Proceedings of the 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. 295–300.
- [17] Hao Li, Haoxiang Zhang, and Ahmed E Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv preprint arXiv:2507.15003* (2025).
- [18] Zachary C Lipton, Charles Elkan, and Balakrishnan Naryanaswamy. 2014. Optimal thresholding of classifiers to maximize F1 measure. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 225–239.
- [19] Guillermo Macbeth, Eugenia Razumiejczyk, and Rubén Daniel Ledesma. 2011. Cliff’s Delta Calculator: A non-parametric effect size program for two groups of observations. *Universitas Psychologica* 10, 2 (2011), 545–555.
- [20] Mary L McHugh. 2013. The chi-square test of independence. *Biochemia medica* 23, 2 (2013), 143–149.
- [21] Patrick E McKnight and Julius Najab. 2010. Mann-whitney U test. *The Corsini encyclopedia of psychology* (2010), 1–1.
- [22] Daniel Ogenrwt and John Businge. 2025. PatchTrack: A Comprehensive Analysis of ChatGPT’s Influence on Pull Request Outcomes. *arXiv preprint arXiv:2505.07700* (2025).
- [23] OpenAI. 2025. Introducing GPT-5.2. <https://openai.com/index/introducing-gpt-5-2/>. Accessed: 2025-12-12.
- [24] Qwen Team. 2024. Qwen3-Embedding-0.6B. <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.
- [25] Deepak-George Thomas, Matteo Biagiola, Nargiz Humbatova, Mohammad Wardat, Gunel Jahangirova, Hridesh Rajan, and Paolo Tonella. 2025. muPRL: A Mutation Testing Pipeline for Deep Reinforcement Learning based on Real Faults. *ICSE* (2025).
- [26] Uraz Cengiz Türker, Robert M Hierons, Khaled El-Fakih, Mohammad Reza Mousavi, and Ivan Y Tyukin. 2024. Accelerating finite state machine-based testing using reinforcement learning. *IEEE Transactions on Software Engineering* 50, 3 (2024), 574–597.
- [27] Lukas Twist, Jie M Zhang, Mark Harman, and Helen Yannakoudakis. 2025. Library Hallucinations in LLMs: Risk Analysis Grounded in Developer Queries. *arXiv preprint arXiv:2509.22202* (2025).
- [28] Guoqing Wang, Zeyu Sun, Jinhao Dong, Yuxia Zhang, Mingxuan Zhu, Qingyuan Liang, and Dan Hao. 2025. Is It Hard to Generate Holistic Commit Message? *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–28.
- [29] Huaning Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. 2025. Ai agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126* (2025).
- [30] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E Hassan. 2025. On the use of agentic coding: An empirical study of pull requests on github. *arXiv preprint arXiv:2509.14745* (2025).
- [31] Fengcai Wen, Csaba Nagy, Gabriele Bavota, and Michele Lanza. 2019. A large-scale empirical study on code-comment inconsistencies. In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*. IEEE, 53–64.
- [32] Marvin Wyrich, Raoul Ghit, Tobias Haller, and Christian Müller. 2021. Bots don’t mind waiting, do they? Comparing the interaction with automatically and manually created pull requests. In *2021 IEEE/ACM Third International Workshop on Bots in Software Engineering (BotSE)*. IEEE, 6–10.
- [33] Qingyu Zhang, Puzhuo Liu, Peng Di, and Chenxiong Qian. 2025. CodeFuse-CommitEval: Towards Benchmarking LLM’s Power on Commit Message and Code Change Inconsistency Detection. *arXiv preprint arXiv:2511.19875* (2025).
- [34] Xunhui Zhang, Yue Yu, Georgios Gousios, and Ayushi Rastogi. 2022. Pull request decisions explained: An empirical overview. *IEEE Transactions on Software Engineering* 49, 2 (2022), 849–871.